Expression Aliases

Document #:	P2826R3
Date:	2026-05-12
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Gašper Ažman <gasper.azman@gmail.com>

Contents

§ 1 Introduction

This paper introduces a way to rewrite a function call to a different expression without a forwarding layer.

Example:

```
int g(int) { return 42; }
int f(long) { return 43; }

using g(unsigned x) = (f(x)); // handle unsigned int with f(long)

int main() {
    g(1); // 42
    g(1u); // 43
}
```

One might think that declaring

```
int g(unsigned x) { return f(x); }
```

is equivalent; this is far from the case in general, which is exactly why we need this capability. See the motivation section for the full list of issues this capability solves.

First, however, this paper introduces what it *does*.

§ 2 Status

This paper has been discussed in EWGi and forwarded to EWG.

§ 3 Proposal

We propose a new kind of entity, called an **expression alias**, of the form:

alias-declaration: `using` *declarator* `=` `(` *expression* `)` `;`

The *declarator* must be a function declarator. The parameters of the function declarator are in scope within the *expression*.

We call the expression evaluated the **target expression*.

Such a definition has to be the first declaration of the declarator.

A call expression that resolves to an expression alias instead resolves to the evaluation of the target expression. The parameter names in the target expression refer directly to the expressions bound as arguments, without any interceding conversions that might have been synthesized in order to perform overload resolution.

This may render the program ill-formed if the substituted expression is ill-formed.

Notes:

— Expression aliases participate in overload resolution the standard way, but do not introduce a new scope and have **no linkage** (they exist purely for the purposes of

overload resolution).

— This feature is *not* the same as calling the target function from the body (see the example with immovable types). It acts as a transparent redirect.

— **SFINAE vs. Hard Errors**: The expression after the `=` is not part of the immediate context. If the target expression is invalid after substitution, it results in a hard error. Conditionally disabling an alias should be done via a `requires` clause on the declaration itself.

§ 3.1 Access Control

Expression aliases respect access control and should be able to friend or call private functions, exactly as if the alias expression were in the scope of a member function.

§ 3.2 Constant Evaluation Check

Adding `constexpr` to the parameters (e.g., `using fn(constexpr int a) = (expr);`) could act as a mechanism to verify that the expression can be constant-evaluated ahead of time, rather than resulting in a hard failure later.

§ 3.3 Examples

§ 3.3.1 Example 1: simple free function aliases

We could implement overload sets from the C library without indirection:

```
// <cmath>
using exp(float x)       = (::expf(x)); // no duplication
double exp(double)      { /* normal definition */ }
using exp(long double x) = (::expl(x)); // no duplication
```

This capability would make wrapping C APIs much easier, since we could just make overload sets out of individually-named functions.

§ 3.3.2 Example 2: simple member function aliases

```
template <typename T>
struct Container {
  auto cbegin() const -> const_iterator;
  auto begin() -> iterator;
  using begin() const = (cbegin()); // saves on template instantiations
};
```

§ 3.3.3 Example 3: avoiding template bloat

`fmt::format` goes through great lengths to validate format string compatibility at compile time, but *really* does not want to generate code for every different kind of string.

This is extremely simple to do with this extension - just funnel all instantiations to a `string_view` overload.

```
template <typename FmtString, typename... Args>
    requires compatible<FmtString, Args...>
using format(FmtString const& fmt, Args const&... args)
    = (vformat(std::string_view(fmt), args...));
```

Contrast with the best we can realistically do presently:

```
template <typename FmtString, typename... Args>
    requires compatible<FmtString, Args...>
auto format(FmtString const& fmt, Args const&... args) -> std::string {
    return format(std::string_view(fmt), args...);
}
```

The second example results in a separate function for each format string (which is, say, one per log statement). The expression alias provably never instantiates different function bodies for different format strings.

§ 3.3.4 Example 4: deduce-to-baseclass

Imagine we inherited from `std::optional`, and we wanted to forward `operator*`, but have it be treated the same as `value()`. This becomes trivial:

```
template <typename T>
struct my_optional : std::optional<T> {
    using operator*(this auto&& self)
          = (*static_cast<copy_cvref_t<decltype(self), std::optional<T>>>
        (self));
};
```

§ 3.3.5 Example 5: immovable argument types

Consider having an argument type that must be in-place constructed:

```
#include <type_traits>
#include <utility>

template <typename T>
struct pin {
    T value;

    pin(auto&&... vs)
        requires(requires { T(std::forward<decltype(vs)>(vs)...); })
        : value(std::forward<decltype(vs)>(vs)...) {}
    pin(pin&&) = delete;
};

template <typename T>
void takes_pinned(pin<T> x) {}

template <typename T>
void takes_pinned_adapter(pin<T> x) { // oops
```

```
    takes_pinned(std::forward<decltype(x)>(x));
}

template <typename T>
using    takes_pinned_alias(T&&    x)    =    (takes_pinned<std::decay_t<T>>
        (std::forward<T>(x)));

int main() {
    takes_pinned<int>(3); // (A) OK; what we have to write
    takes_pinned(3);      // Error, but what we want to write
    takes_pinned(pin<int>{3}); // OK, but verbose
    takes_pinned_adapter(pin<int>{3}); // Error, can't forward pin<T>
    takes_pinned_alias(3); // OK with this paper, same as (A)
}
```

§ 3.3.6 Example 6: Customization Point Objects (CPOs)

Expression Aliases make CPOs behave seamlessly without overhead. For example, `std::ranges::begin` could simply be an alias:

```
namespace __adl_protected {
  template <typename T>
  concept adl_test_begin = requires (T t) { begin(t); };
  template <typename T>
  using adl_begin(T&& t) = (begin(t));
}
namespace std::ranges {
struct begin_t {
  // objects that support x.begin(); omitting the "and that's an iterator"
        clause
  template <typename T>
  using operator()(this begin_t, T&& t) const
    requires /* requires clause for "any one of the rules work" */
    // using [P2806R3]; do-expressions
    = do {
      if constexpr(enable_borrowed_range<remove_cv_t<T>>) {
        // If E is an rvalue and enable_borrowed_range<remove_cv_t<T>>
        // is false, ranges::begin(E) is ill-formed.
        static_assert(false, "T does not enable borrowing");
      } else if constexpr (is_array_v<remove_cv_t<T>>) {
        if constexpr(__is_complete(remove_all_extents<remove_cv_t<T>)) {
                  // Otherwise, if T is an array type (9.3.4.5) and
        remove_all_extents_t<T> is an incomplete type,
          // ranges::begin(E) is ill-formed with no diagnostic required.
            static_assert(false, "Can't begin() over arrays of incomplete
        types");
        } else {
              // Otherwise, if T is an array type, ranges::begin(E) is
        expression-equivalent to t + 0
          do_return t + 0;
        }
      } else if constexpr(requires { t.begin(); } &&
                                                          requires   {
        input_or_output_iterator<decltype(t.begin())>}) {
         //Otherwise, if auto(t.begin()) is a valid expression whose type
        models
```

```
            // input_or_output_iterator, ranges::begin(E) is expression-
        equivalent to auto(t.begin())
          do_return auto(t.begin());
        } else if constexpr(requires { __adl_protected::adl_begin(t); } &&
                                                                requires {
          input_or_output_iterator<decltype(__adl_protected::adl_begin(t))>})
          {
                //Otherwise, if T is a class or enumeration type and
        auto(begin(t)) is a valid expression whose type
           // models input_or_output_iterator where the meaning of begin is
        established as-if by performing
           // argument-dependent lookup only (6.5.4), then ranges::begin(E)
        is expression-equivalent to that
          // expression.
          do_return auto(__adl_protected::adl_begin(t));
        } else {
          // Otherwise, ranges::begin(E) is ill-formed.
          static_assert(false, "Can't begin()");
        }
      };
};
inline constexpr begin = begin_t{};
} // end namespace
```

§ 3.3.7 Example 8: Literal checking on strings

```
struct cstring_view {
  // ..
  template <size_t N>
          using     cstring_view(const     char     (&in)[N])     requires
         (std::meta::is_literal(^^in))
       = (cstring_view(in, N - 1));
  // ...
};
```

§ 3.3.8 Example 9: Constant evaluation verification constraint

```
template <auto V> constexpr auto constant = V;
using as_constant(auto X) requires (X, true) = (constant<X>);
```

§ 3.3.9 Example 10: Trivial Library Extensions for Safety

The standard library replaced `operator>>(istream&, char*)` with `operator>>(istream&, char(&)[N])` for obvious safety reasons. Adding support for `std::array` or `std::span` to benefit from the same safety would be trivial and avoid new instantiations of the actual I/O logic.

For instance, we can forward `std::array` directly to the safe C-array overload by aliasing the underlying array member:

```
namespace std {
  template <class charT, class traits, size_t N>
  using operator>>(basic_istream<charT, traits>& is, array<charT, N>& arr)
```

```
      = (is >> arr._M_elems); // dispatches directly to the safe C-array
        overload
}
```

Alternatively, we can use an alias to transparently bridge contiguous containers to a `std::span` implementation. If we define the actual I/O logic for a span of a specific size, an alias can perform the conversion without introducing an intermediate function frame. The key benefit here is that `std::span`'s constructor deduces the `N` parameter from the container, naturally and efficiently routing the call to the correctly-sized template overload:

```
// 1. The actual implementation:
template <size_t N>
std::istream& operator>>(std::istream& is, std::span<char, N> spn) { /*
        ... */ }

// 2. An alias that accepts containers (like std::array) and forwards to
        the span overload:
using operator>>(std::istream& is, auto&& range)
    requires requires { std::span(std::forward<decltype(range)>(range)); }
    = (is >> std::span(std::forward<decltype(range)>(range)));

void test() {
    std::array<char, 42> buffer;
    std::cin >> buffer; // Picks the alias, converts to span, and invokes
        (1)
}
```

§ 3.3.10 Example 11: String-like Types and `string_view`

A similar situation arises with all string-like types and `std::basic_string_view`. The standard library implements `operator<<` for `std::basic_string_view` by deducing `CharT` and `Traits`.

Currently, if you implement a custom string type (such as a compile-time `fixed_string`), you often must write a wrapper `operator<<` that explicitly converts your type to `std::string_view` and then delegates the call to avoid ambiguity or missing overloads. Expression aliases eliminate this boilerplate:

```
template <class CharT, size_t N>
struct fixed_string {
    /* ... */
    constexpr operator std::basic_string_view<CharT>() const;
};

// A single expression alias transparently bridges any string-like type
// to the basic_string_view implementation:
using operator<<(std::ostream& os, auto const& str)
    requires requires { std::basic_string_view(str); }
    = (os << std::basic_string_view(str));
```

§ 4 Why this syntax

— The `using` syntax aligns with type aliases.

— The function declarator naturally introduces names for bound expressions and their substitution.

— It seamlessly represents the pure forwarding nature without instantiating extra template scopes.

§ 5 Nice-to-have properties

— **Copy Elision & Programmable Dispatch**: Enables programmable dispatch without actual argument binding, allowing the compiler to cleanly preserve copy elision (conversions are thrown away and recalculated within the expression)

— **Removal of library-defined dispatch from callstacks**: the library defines many customization point objects as "expression-equivalent to *list-of-cases*". This feature should be able to make all of them truly act that way.

— **Optimizers have to work way less hard**: the extra inline-ing contexts are hard on optimizers; inlining depth is a valuable resource, and the intermediate symbols still get emitted, optimized, and then sometimes garbage-collected.

— **Major debug symbol reduction**: expression aliases don't need cross-TU mangling and have no linkage.

— **No extra work for overload resolution** - this feature interacts as-if it were a normal function for the purposes of overload resolution. No new rules required.

§ 6 Related Work

Roughly related were Parametric Expressions [P1221R1], but they didn't interact with overload sets very well.

This paper is strictly orthogonal, as it doesn't give you a way to rewire the arguments at the language level in a new way, just substitute the expression that's actually invoked.

§ 7 Use-cases

There are many cases in C++ where we want to add a function to an overload set without wrapping it.

A myriad of use-cases are in the standard itself, just search for *expression_equivalent* and you will find many.

The main thing we want to do in all those cases is:

— step 1: detect a call expression pattern

— step 2: dispatch to the appropriate function for that case

The problem is that the detection of the pattern and the appropriate function to call often don't fit into the standard overload resolution mechanism of C++, which necessitates a wrapper, such as a customization point object, or some kind of dispatch function.

This dispatch function, however it's employed, has no way to distinguish prvalues from other rvalues and therefore cannot possibly emulate the copy-elision aspects of *expression-equivalent*. It is also a major source of template bloat.

§ 7.1 deduce-to-type

The problem of template-bloat when all we wanted to do was deduce the type qualifiers gets much worse with the introduction of (Deducing This) [P0847R7] into the language.

This topic is explored in Barry Revzin's [P2481R1].

Observe how easy it is to forward with an explicit-object member function using expression aliases:

```
struct C {
  void f(this std::same_as<C> auto&& x) {} // implementation
  template <typename T>
  using f(this T&& x) = (static_cast<copy_cvref_t<T, C>&&>(x).f());
};
struct D : C {};

void use_member() {
  D d;
  d.f();                // OK, calls C::f(C&)
  std::move(d).f();     // OK, calls C::f(C&&)
  std::as_const(d).f(); // OK, calls C::f(C const&)
}
```

§ 7.1.1 Don't lambdas make this shorter?

While one could use an inline lambda, lambdas still depend on `T`, since one can use it in the lambda body, leading to more instantiations. Expression aliases sidestep this by just rewriting the call.

§ 7.2 The "Rename Overload Set" / "Rename Function" refactoring becomes possible in large codebases

In C++, "rename function" or "rename overload set" are not refactorings that are physically possible for large codebases without at least temporarily risking overload resolution breakage.

However, with this paper, one can disentangle overload sets by leaving removed overloads where they are, and redirecting them to the new implementations, until they are ready to be removed.

(*Reminder:* conversions to reference can lose fidelity, so trampolines in general do not work).

One can also just define

```
using new_name(auto&&...args)
    requires requires { old_name(std::forward<decltype(args)>(args)...); }
    = (old_name(std::forward<decltype(args)>(args)...));
```

and have a new name for an old overload set.

§ 7.3 The "rename function" refactoring becomes ABI stable

We can finally move overload sets around and not break ABI in some cases since we basically gain true function aliases.

§ 8 FAQ

§ 8.1 What if the chosen expression alias has a failing requires clause?

It drops out of overload resolution cleanly, they way a function would. If it passes the `requires` clause but the substituted expression is ill-formed, it is a hard error.

§ 9 Acknowledgements

— Hana Dusíková for implementing the paper and many discussions, comments, and collaboration.

— Barry Revzin for his comments and work on reflection and code injection.

— Daveed Vandevoorde for lots of early guidance.

§ 10 References

[P0847R7] Deducing this.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html

[P1221R1] Parametric Expressions.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1221r1.html

[P2481R1] Forwarding reference to specific type/template.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2481r1.html